



TRATAMENTO DE DADOS E COMPRESSÃO

Prof. Dr. Julio César Santos dos Anjos
Dados e Informação

Técnicas de Compressão de Dados

— 27 de maio de 2026

1. Taxa de Informação
2. Codificação de Fonte
3. Compressão de Dados
4. Capacidade de Canal



1. Taxa de Informação
2. Codificação de Fonte
3. Compressão de Dados
4. Capacidade de Canal



Taxa de Informação - Conceito Geral

✓ Em computação

- A taxa de informação (ou taxa de transferência/dados) refere-se à quantidade de dados transferidos por unidade de tempo, geralmente medida em bits por segundo (bps).
- Ela define a velocidade de movimentação de dados entre dispositivos, redes ou armazenamento, influenciando diretamente o desempenho do sistema;

✓ Em telecomunicação

- É a taxa de código (ou taxa de informação) de um código de correção adiantada de erros. Também definida como a proporção do stream que é útil (não redundante).



No Contexto da Agregação de Informação

A taxa de informação de uma fonte é determinada a partir de suas entropia e taxa de transmissão.

- ✓ Seja uma fonte de informação S que transmite r símbolos a cada segundo, e estes símbolos são variáveis aleatórias a_s pertencentes ao alfabeto $\mathcal{A} = \{a_s : 1 \leq s \leq S\}$, defini-se a taxa de informação R como:

$$R = r \cdot \mathcal{H}(\mathcal{A}) \quad (81)$$



Aspéctos da Informação $\mathcal{H}$ e $\mathcal{I}$

- ✓ Entropia e informação mútua são importantes características de processos aleatórios com possibilidade de aplicação em várias áreas.
- ✓ Um problema inerente ao tratamento por teoria da estimação é como estimar tais quantidades uma vez que elas são funções da densidade de probabilidade, a qual é difícil de estimar a partir dos dados.



Aspéctos da Informação $\mathcal{H}$ e $\mathcal{I}$

- ✓ Entropia e informação mútua são importantes características de processos aleatórios com possibilidade de aplicação em várias áreas.
- ✓ Um problema inerente ao tratamento por teoria da estimação é como estimar tais quantidades uma vez que elas são funções da densidade de probabilidade, a qual é difícil de estimar a partir dos dados.

Como então podemos fazer estimativas dos dados a partir de somente medidas?



Aspéctos da Informação $\mathcal{H}$ e $\mathcal{I}$

- ✓ Entropia e informação mútua são importantes características de processos aleatórios com possibilidade de aplicação em várias áreas.
- ✓ Um problema inerente ao tratamento por teoria da estimação é como estimar tais quantidades uma vez que elas são funções da densidade de probabilidade, a qual é difícil de estimar a partir dos dados.

Como então podemos fazer estimativas dos dados a partir de somente medidas?

Usando Estimadores!!!!



Estimação de Gram-Charlier

- ✓ Considerando que pela aproximação polinomial da densidade de probabilidade em torno de uma pdf gaussiana, por Gram-Charlier

$$p_X(x) = p_G(x) \left(1 + \sum_{K=3}^{\infty} C_k \cdot \mathfrak{h}_i(x) \right) \quad (82)$$

- ✓ Os coeficientes C_k serão funções dos momentos/cumulantes de X e $\mathfrak{h}_i$ é o polinômio de Hermite* de ordem k .

*Os polinômios de Hermite $\mathfrak{h}_i$ são um conjunto de polinômios ortogonais no domínio $(-\infty, \infty)$ com função de ponderação $e^{(-x^2)}$. Definido como a integral de contorno.



Estimação de Gram-Charlier

- ✓ Sabendo que podemos escrever a entropia como

$$\mathcal{H}(p_X) = \mathcal{H}(p_G) - \mathcal{N}_G(p_X) \quad (83)$$

- ✓ Em que $\mathcal{N}_G(p_X)$ é a negentropia
- ✓ Assim podemos reescrever a entropia como

$$\mathcal{H}(p_X) = \mathcal{H}(p_G) - \int_V p_X(\mathbf{v}) \log \left[\frac{p_X(\mathbf{v})}{p_G(\mathbf{v})} \right] d\mathbf{v} \quad (84)$$



Estimação de Gram-Charlier

✓ Então

$$\begin{aligned}\mathcal{H}(p_X) &\approx \mathcal{H}(p_G) - \int_V p_X(\mathbf{v})(1 + Z(\mathbf{v})) \log [1 + Z(\mathbf{v})] d\mathbf{v} \\ &\approx \mathcal{H}(p_G) - \int_V p_X(\mathbf{v}) [Z(\mathbf{v}) + Z^2(\mathbf{v})] d\mathbf{v}\end{aligned}\quad (85)$$

- ✓ Considerando que $Z(\mathbf{v}) = \frac{1}{3!} \sum_{i,j,k} K^{i,j,k}(\mathbf{v}) \mathfrak{h}_{i,j,k}(\mathbf{v})$, $\mathfrak{h}_{i,j,k}$ é o polinômio de Hermite de ordem i, j, k e $K^{i,j,k}$ é o momento de terceira ordem, então

$$\mathcal{H}(p_X) \approx \mathcal{H}(p_G) - \frac{1}{12} \left[\sum_{i=1}^d (K^{i,i,i})^2 + 3 \sum_{\substack{i,j=1 \\ i \neq j}}^d (K^{i,i,j})^2 + \frac{1}{6} \sum_{\substack{i,j,k=1 \\ i \neq j \\ j \neq k}}^d (K^{i,j,k})^2 \right] \quad (86)$$



Estimação de Gram-Charlier

- ✓ É importante observar que foi feita uma aproximação em série, de segunda ordem, na expansão de Edgeworth (expansão de Gram-Charlier ordenada pela ordem de importância dos termos).
- ✓ Os momentos podem então ser estimados a partir das amostras
- ✓ Os polinômios de Hermite têm forma fechada e pode ser calculados a partir dos dados.
- ✓ Entretanto, as expansões de Edgeworth e Gram-Charlier só podem aproximar funções que são “**próximas**” a uma gaussiana



Estimação de Parzen

- ✓ O estimador de Parzen para pdfs aproxima a densidade (qualquer!) por um somatório de funções kernel. Por ex.: a função gaussiana
- ✓ Pode-se dizer que

$$p_X(\mathbf{x}) = \frac{1}{N} \sum_{i=1}^N \mathcal{K}(\mathbf{x} - \mathbf{x}_i, \sigma_i \mathbf{I}) \quad (87)$$

- ✓ Já para dois kernels vale a seguinte expressão

$$\int_{\mathbf{x}} \mathcal{K}(\mathbf{x} - \mathbf{x}_i, \sigma_1 \mathbf{I}) \mathcal{K}(\mathbf{x} - \mathbf{x}_j, \sigma_2 \mathbf{I}) d\mathbf{x} = \mathcal{K}(\mathbf{x}_i - \mathbf{x}_j, (\sigma_1 + \sigma_2) \mathbf{I}) \quad (88)$$



Estimação de Parzen

- ✓ Portanto, poderemos ter a seguinte **aproximação para a Entropia de Shannon**

$$\begin{aligned}\mathcal{H}(p_X) &= - \int_{\mathbf{x}} p_X(\mathbf{x}) \cdot \log(p_X(\mathbf{x})) d\mathbf{x} \\ &= - \int_{\mathbf{x}} \sum_{i=1}^N \mathcal{K}(\mathbf{x} - \mathbf{x}_i) \cdot \log \left[\sum_{i=1}^N \mathcal{K}(\mathbf{x} - \mathbf{x}_i) \right] d\mathbf{x} \quad (89)\end{aligned}$$



Estimação de Parzen

✓ Para a **entropia de Rényi** temos para o caso de $\alpha = 2$

$$\begin{aligned}\mathcal{H}(p_X) &= -\log \left[\int_{\mathbf{x}} p_X^2(\mathbf{x}) d\mathbf{x} \right] \\ &\approx -\log \left[\frac{1}{N^2} \int_{\mathbf{x}} \sum_{i=1}^N \sum_{j=1}^N \mathcal{K}(\mathbf{x} - \mathbf{x}_i, \sigma \mathbf{I}) \mathcal{K}(\mathbf{x} - \mathbf{x}_j, \sigma \mathbf{I}) d\mathbf{x} \right] \\ &\approx -\log \left[\frac{1}{N^2} \sum_{i=1}^N \sum_{j=1}^N \mathcal{K}(\mathbf{x}_i - \mathbf{x}_j, 2\sigma \mathbf{I}) \right] d\mathbf{x} \quad (90)\end{aligned}$$



1. Taxa de Informação
2. Codificação de Fonte
3. Compressão de Dados
4. Capacidade de Canal

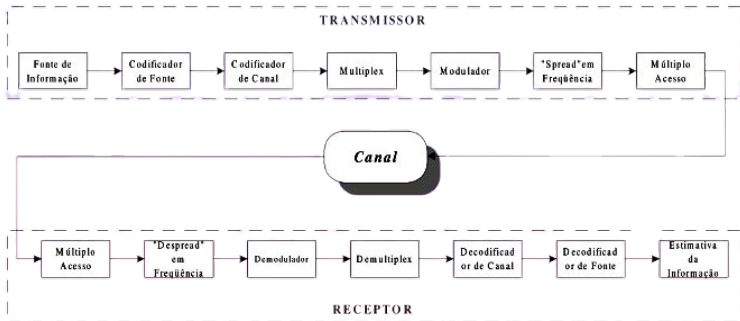


Considerações

- ✓ Codificação de fonte é a representação dos dados da fonte de forma eficiente
- ✓ O dispositivo responsável por esta tarefa é chamado codificador de fonte
- ✓ Tipos de código
 - Símbolos mais freqüentes com palavras menores
 - Símbolos menos freqüentes (raros) com palavras maiores
- ✓ Códigos de tamanho variável
 - Exemplo: código Morse (língua inglesa)
Letra mais freqüente: E (' . ')
Letra menos freqüente: Q (' - - . - ')

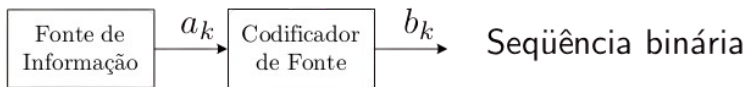


Modelo Geral do Sistema de Comunicação



Codificador de fonte

- ✓ Palavras código são dadas na forma binária;
- ✓ O código da fonte é **unicamente decodificável** tal que a sequência de símbolos original possa ser reconstruída perfeitamente por meio da sequência binária codificada;



Codificador de fonte - Hipóteses (I)

1. Sequência da fonte de informação possui K diferentes símbolos;
 2. Probabilidade de ocorrência do k -ésimo símbolo (a_k) é denominada p_k ;
 3. A palavra código (binária) associada ao símbolo a_k tem tamanho l_k ;
- ✓ Assim, o comprimento médio da palavra código (número médio de bits por símbolo da fonte usado na codificação) pode ser dado por:

$$L = \sum_{k=0}^{K-1} p_k \cdot l_k \quad (91)$$



Codificador de fonte - Hipóteses (II)

- ✓ Se o valor mínimo possível de L : $L_{\min}$
- ✓ A **Eficiência de codificação** do codificador da pode ser dada por:

$$\eta = \frac{L_{\min}}{L} \quad (92)$$

- ✓ O codificador é dito eficiente quando $\eta \rightarrow 1$



Codificador de fonte - Hipóteses (II)

- ✓ Se o valor mínimo possível de $L : L_{\min}$
- ✓ A **Eficiência de codificação** do codificador da pode ser dada por:

$$\eta = \frac{L_{\min}}{L} \quad (92)$$

- ✓ O codificador é dito eficiente quando $\eta \rightarrow 1$

Mas, como determinar o valor $L_{\min}$?



1º Teorema de Shannon : Teorema da codificação de fonte

- ✓ Também chamado de *Teorema da codificação sem ruído* - trata da condição de codificação sem erros;
- ✓ Responde a questão fundamental da codificação de fonte;

Teorema 1. Dada uma fonte de informação discreta com entropia $\mathcal{H}(A)$, o tamanho médio da palavra código L para qualquer codificação de fonte sem distorção é limitado por

$$L \geq \mathcal{H}(A) \quad (93)$$



Codificação de Fonte - limites

Entropia da Fonte

Limite fundamental no número médio de bits para representar cada símbolo da fonte

Limite mínimo

$$L_{\min} = \mathcal{H}(A) \quad (94)$$

Eficiência da Codificação

$$\eta = \frac{\mathcal{H}(A)}{L} \quad (95)$$



1. Taxa de Informação
2. Codificação de Fonte
3. Compressão de Dados
4. Capacidade de Canal



O que fazer para achar o código?

- ✓ Remoção da redundância de informação do sinal a ser transmitido;
- ✓ Processo geralmente chamado de compactação de dados ou compressão sem perdas;



O que fazer para achar o código?

- ✓ Remoção da redundância de informação do sinal a ser transmitido;
- ✓ Processo geralmente chamado de compactação de dados ou compressão sem perdas;

Como gerar códigos com comprimento médio próximo da entropia?



Codificação Prefixo

- ✓ Restrição de codificação de fonte: ser unicamente decodificável;
- ✓ Nenhuma sequência de palavras código correspondente a uma dada sequência da fonte pode ser associada a outra sequência qualquer;

Código Prefixo

É um código no qual nenhuma palavra código é prefixo de qualquer outra palavra código.



Codificação Prefixo

Símbolo da fonte	Probabilidade de ocorrência	Código I	Código II	Código III
a_0	0.5	0	0	0
a_1	0.25	1	10	01
a_2	0.125	00	110	011
a_3	0.125	11	111	0111

Código II é um código prefixo!

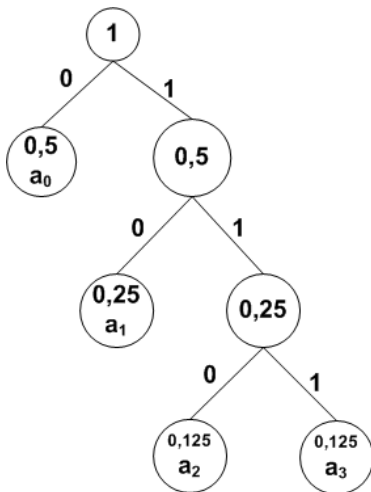


Decodificação - Código Prefixo

- ✓ Busca numa **árvore de decisão**
- ✓ Facilidade de decodificação
- ✓ Cada bit que é recebido move o detector para algum ponto da árvore e um respectivo símbolo da fonte
- ✓ São sempre unicamente decodificáveis



Árvore de Decisão - Código Prefixo



Códigos Prefixo

- ✓ Fonte discreta sem memória: $\{a_0, a_1, \dots, a_{k-1}\}$
- ✓ Probabilidades dos eventos: $\{p_0, p_1, \dots, p_{k-1}\}$
- ✓ Tamanho das palavras código: $\{l_0, l_1, \dots, l_{k-1}\}$

Inequação de Kraft-McMillan

O tamanho das palavras códigos de um código prefixo devem satisfazer a seguinte inequação

$$\sum_{k=0}^{k-1} 2^{-l_k} \leq 1 \quad (96)$$



Códigos Prefixo

- ✓ Usando o inverso da definição da inequação de Kraft-McMillan pode-se dizer que
Se os tamanhos das palavras código de um determinado código respeitam a inequação de Kraft-McMillan, então um código prefixo com aquelas palavras código pode ser construído
- ✓ Embora códigos prefixos sejam unicamente decodificáveis, o inverso não é verdade
- ✓ Outros códigos unicamente decodificáveis não permitem conhecer sempre o fim de uma palavra código
- ✓ Códigos prefixo são também chamados de **códigos instantâneos**



Códigos Prefixo - Equacionamento

- ✓ Seja uma fonte discreta sem memória com entropia $\mathcal{H}(A)$, o tamanho médio da palavra código de um código prefixo é limitado por

$$\mathcal{H}(A) \leq L \leq \mathcal{H}(A) + 1 \quad (97)$$

- ✓ O lado esquerdo de (97) é satisfeito com a igualdade sob a condição de

$$p_k = 2^{-l_k} \quad (98)$$

- ✓ Portanto

$$\sum_{k=0}^{k-1} 2^{-l_k} \leq \sum_{k=0}^{k-1} p_k = 1 \quad (99)$$



Códigos Prefixo - Equacionamento

- ✓ Usando-se a inequação de Kraft-McMillan (construção do código), o tamanho médio da palavra código é

$$L = \sum_{h=0}^{k-1} \frac{l_k}{2^{l_k}} \quad (100)$$

- ✓ A Entropia da fonte então

$$\begin{aligned} \mathcal{H}(A) &= - \sum_{h=0}^{k-1} \left(\frac{1}{2^{l_k}} \right) \log \left(2^{-l_k} \right) \\ &= \sum_{h=0}^{k-1} \left(\frac{l_k}{2^{l_k}} \right) \end{aligned} \quad (101)$$



Códigos Prefixo - Equacionamento

- ✓ Usando-se a inequação de Kraft-McMillan (construção do código), o tamanho médio da palavra código é

$$L = \sum_{h=0}^{k-1} \frac{l_k}{2^{l_k}} \quad (100)$$

- ✓ A Entropia da fonte então

$$\begin{aligned} \mathcal{H}(A) &= - \sum_{h=0}^{k-1} \left(\frac{1}{2^{l_k}} \right) \log \left(2^{-l_k} \right) \\ &= \sum_{h=0}^{k-1} \left(\frac{l_k}{2^{l_k}} \right) \end{aligned} \quad (101)$$

Neste caso, **o Código Prefixo apresenta a codificação mais eficiente!**



Códigos Prefixo - Equacionamento

- ✓ Como encontrar o código prefixo para uma fonte arbitrária (probabilidades de ocorrência dos símbolos quaisquer valores)?
- ✓ Resposta: uso de código estendido.
- ✓ Seja L_n o tamanho médio da palavra código de um código prefixo estendido. Para um código unicamente decodificável L_n é o menor possível.

$$\begin{aligned}\mathcal{H}(A^n) &\leq L_n < \mathcal{H}(A^n) + 1 \\ n \cdot \mathcal{H}(A) &\leq L_n < n \cdot \mathcal{H}(A) + 1 \\ \mathcal{H}(A) &\leq \frac{L_n}{n} < \mathcal{H}(A) + \frac{1}{n}\end{aligned}\tag{102}$$

- ✓ No limite, os limites inferior e superior convergem

$$\lim_{n \rightarrow \infty} \frac{L_n}{n} = \mathcal{H}(A)\tag{103}$$



Códigos Prefixo - Equacionamento

- ✓ Então, fazendo a ordem n de um código prefixo estendido grande o suficiente, o código pode representar, fidedignamente, uma fonte A tão próximo quanto se deseje;
- ✓ Ou seja, o tamanho médio de uma palavra código de um código estendido pode ser tão pequeno quanto a entropia da fonte, dado que o código estendido tem um tamanho suficiente;
- ✓ **Porém, o custo de complexidade de decodificação: aumenta na ordem do tamanho do código estendido.**



Considerações para o Codificador de Huffman

- ✓ Idéia básica: associar a cada símbolo, uma sequência de bits, aproximadamente igual a quantidade de informação existente no símbolo
- ✓ Tamanho médio da palavra código aproxima o limite (entropia)
- ✓ Substituição do conjunto de estatísticas da fonte por um conjunto **mais simples**
- ✓ Processo de redução iterativo até que o restem apenas dois símbolos para os quais '0' e '1' é um código ótimo
- ✓ Construção do código é feita através da retropropagação



Algoritmo de Huffman

1. Os símbolos da fonte são listados em ordem decrescente das probabilidades. Os dois símbolos com as menores probabilidades são assinalados como 0 e 1.
2. Os dois símbolos anteriormente assinalados são combinados em um novo símbolo com a soma das probabilidades dos dois primeiros. **A lista decrescente de probabilidades é novamente feita.**
3. Procedimento se repete até sobrares somente dois símbolos.
4. Do final para o início da tabela, a palavra código de cada símbolo é encontrada.



Codificação de Huffman

- ✓ Seja a fonte discreta com as seguintes probabilidades para seus símbolos

Símbolo	Probabilidade
a_0	0.4
a_1	0.2
a_2	0.2
a_3	0.1
a_4	0.1

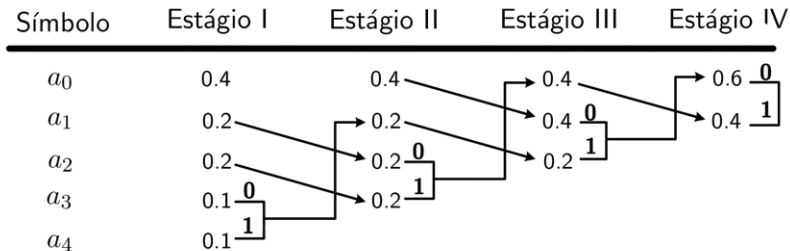
- ✓ Entropia da fonte (Obs.: $\log_2$ por se tratar de bits)
 $\mathcal{H}(A) = 2.12193$

Pergunta

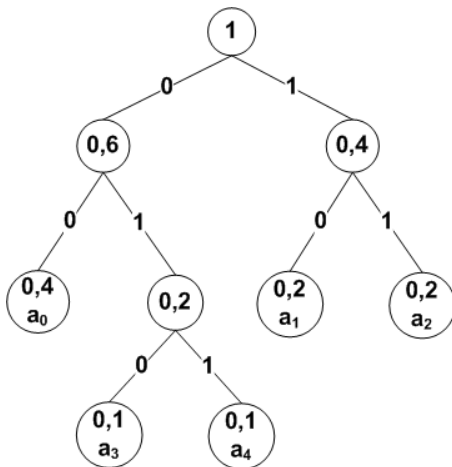
O código de Huffman vai fornecer um tamanho médio da palavra código próximo da entropia da fonte?



Codificação de Huffman



Codificação de Huffman



Codificação de Huffman

Símbolo	Probabilidade	Palavra código
a_0	0.4	00
a_1	0.2	10
a_2	0.2	11
a_3	0.1	010
a_4	0.1	011

✓ Tamanho médio da palavra código

$$L = 2.2 \quad (104)$$

✓ $\mathcal{H}(A)$ de apenas 3.67%

✓ $\mathcal{H}(A) \leq L < \mathcal{H}(A) + 1$



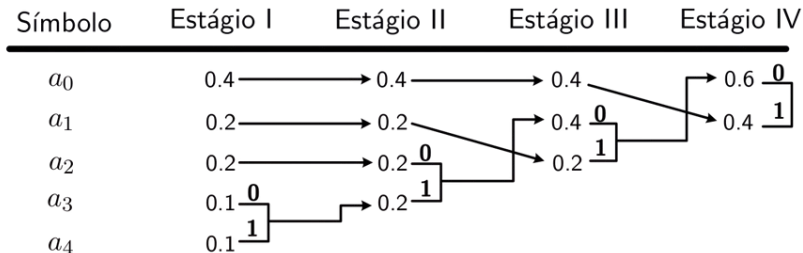
Comentários

- ✓ A codificação de Huffman não é única!
- ✓ Outras maneiras
 1. Arbitrariedade da associação de 0 e 1
 2. Valor da probabilidade igual: colocação dos valores
 3. Diferentes tamanhos médios da palavra código
- ✓ Medida da variabilidade (variância) do tamanho médio das palavras código

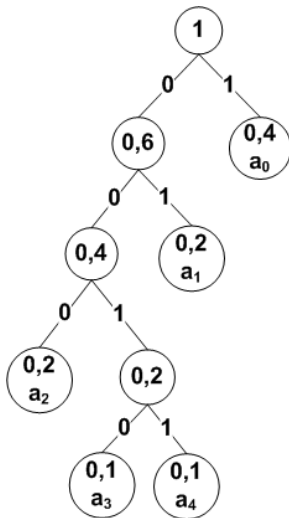
$$\sigma^2 = \sum_{k=0}^{k-1} p_k (l_k - L)^2 \quad (105)$$



Codificação de Huffman - não unicidade



Codificação de Huffman - não unicidade



Codificação de Huffman - não unicidade

Símbolo	Probabilidade	Palavra código
a_0	0.4	1
a_1	0.2	01
a_2	0.2	000
a_3	0.1	0010
a_4	0.1	0011

$$\sigma_1^2 = 0.16$$

$$\sigma_2^2 = 1.36$$



Codificação de Huffman - problemas

- ✓ Requer conhecimento a priori do modelo probabilístico da fonte
- ✓ Na prática, são poucas as situações que se conhece a priori as estatísticas da fonte
- ✓ Além disso, em algumas aplicações, como modelamento de texto, a armazenagem impede o código de Huffman de capturar as relações entre palavras e frase - comprometimento da eficiência do código
- ✓ Quais Alternativa??



Codificação de Lempel-Ziv-Welch

- ✓ Codificação adaptativa por natureza
- ✓ Implementação mais simples que a de Huffman

Idéia

Organizar a sequência de dados da fonte em segmentos que são subsequências menores não encontradas anteriormente.



Codificação de Lempel-Ziv-Welch

1. Seqüência de dados: **000101110010100101**
2. Assume-se que os símbolos '0' e '1' já estão armazenados
3. Início do algoritmo:
Subseqüências armazenadas: 0, 1
Dados para organização: **000101110010100101**
4. Menor seqüência não armazenada?
Subseqüências armazenadas: 0, 1, **00**
Dados para organização: **0101110010100101**
5. Outra seqüência?
Subseqüências armazenadas: 0, 1, 00, **01**
Dados para organização: **01110010100101**
6. Mais uma?
Subseqüências armazenadas: 0, 1, 00, 01, **011**
Dados para organização: **10010100101**



Codificação de Lempel-Ziv-Welch

Tabela de codificação

Pos.:	1	2	3	4	5	6	7	8	9
Subseq.:	0	1	00	01	011	10	010	100	101
Rep. num.:			11	12	42	21	41	61	62
Blocos cod.:			0010	0011	1001	0100	1000	1100	1101



Codificação de Lempel-Ziv-Welch - Palavra código

Algoritmo

1. Inicialize o dicionário para conter todas as strings de comprimento um (geralmente ASCII de 0 a 255).
2. Encontre a string mais longa **W** no dicionário que corresponda à entrada atual.
3. Exiba o índice do dicionário para **W** na saída e remova **W** da entrada.
4. Adicione **W** seguido pelo próximo símbolo na entrada ao dicionário.
5. Se a entrada não terminou. Vá para a Etapa 2.

✓ O método consiste de:

- **compressão**: identificar, na área de pesquisa, a maior sequência de símbolos armazenada no dicionário e substituí-la por um código.
- **descompressão**: os códigos devem ser substituídos pelas sequências de símbolos do dicionário correspondentes.



Principais Características

- ✓ **Sem Perdas:** A descompressão restaura os dados originais exatamente.
- ✓ **Adaptativo:** O dicionário é construído durante a compressão, não precisando ser armazenado com o arquivo.
- ✓ **Código Fixo:** Diferente do Huffman, o LZW geralmente usa códigos de tamanho fixo (ex: 12 bits) para representar sequências variáveis, aumentando o tamanho do código conforme necessário.
- ✓ **Uso:** Ideal para arquivos com alta repetição de padrões, como imagens (GIF/TIFF).
- ✓ Na prática, esse algoritmo consegue um maior nível de compressão de dados que os outros métodos.



Exemplo de Compressão de Lempel-Ziv-Welch

<i>input</i>	A	B	R	A	C	A	D	A	B	R	A	B	R	A	B	R	A
<i>matches</i>	A	B	R	A	C	A	D	A	B		R	A		B	R		A
<i>value</i>	41	42	52	41	43	41	44	81		83		82		88			41 80

LZW compression for A B R A C A D A B R A B R A B R A

key	value
:	:
A	41
B	42
C	43
D	44
:	:

key	value
AB	81
BR	82
RA	83
AC	84
CA	85
AD	86

key	value
DA	87
ABR	88
RAB	89
BRA	8A
ABRA	8B

codeword table



Decodificação de Lempel-Ziv-Welch

- ✓ Ler a entrada **41 42 52 41 43 41 44 81 83 82 88 41 80**
- ✓ Para cada entrada verifique a table com os códigos gerados na compressão,
- ✓ Reconstrua o conteúdo original



1. Taxa de Informação
2. Codificação de Fonte
3. Compressão de Dados
4. Capacidade de Canal



Considerações

- ✓ Codificador de fonte retira redundância - compactação da informação
- ✓ Canal insere perturbações/interferência
- ✓ Necessidade de inserção de redundância (conhecida) para evitar/minimizar os efeitos do canal
- ✓ Transferência de informação torna-se complicada
- ✓ Necessidade de inclusão de algum controle de qualidade na mensagem

Questão

Qual o limite de informação que pode ser transmitida, sem erros, através de um canal que insere perturbação na mensagem?



Teorema da Capacidade de Canal

A capacidade de informação de um canal é a quantidade máxima de informação capaz de ser transmitida, *bits* por uso do *canal*, sem erro de transmissão. Define-se então:

$$C = \max_{p_X(x)} \mathcal{I}(X, Y) \quad (106)$$

- ✓ Capacidade de canal é uma função apenas das probabilidades condicionais $P(y_k|x_j)$ que definem o canal
- ✓ A maximização em relação às probabilidades dos símbolos (codificados) da fonte é sujeita a

1. $P(x_i) \geq 0, \forall i$

2.

$$\sum_{i=0}^{k-1} P(x_i) = 1$$

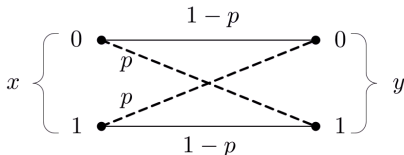


Propriedades da Capacidade de Canal

1. $C \geq 0$ uma vez que $\mathcal{I}(X, Y) \geq 0$
2. $C \leq \log[|X|]$ uma vez que
$$C = \max \mathcal{I}(X, Y) \leq \max \mathcal{H}(X) = \log[|X|]$$
$$C \leq \log[|Y|]$$
3. $\mathcal{I}(X, Y)$ é uma função contínua de $p_X(x)$
4. $\mathcal{I}(X, Y)$ é uma função **côncava** de $p_X(x)$



Canal Binário Simétrico - BSC - Propriedades



- ✓ Diagrama é unicamente definido pela probabilidade de transição p .
- ✓ A entropia é maximizada quando $Pr(x_0) = Pr(x_1) = 1/2$



Capacidade do Canal Binário Simétrico - BSC

✓ $Pr(y_0|x_1) = Pr(y_1|x_0) = p$

✓ $Pr(y_0|x_0) = Pr(y_1|x_1) = 1 - p$

✓ Sabendo

$$\mathcal{I}(X, Y) = \sum_{k=0}^{k-1} \sum_{j=0}^{k-1} P(x_j, y_k) \cdot \log \left[\frac{P(y_k|x_j)}{P(y_k)} \right]$$

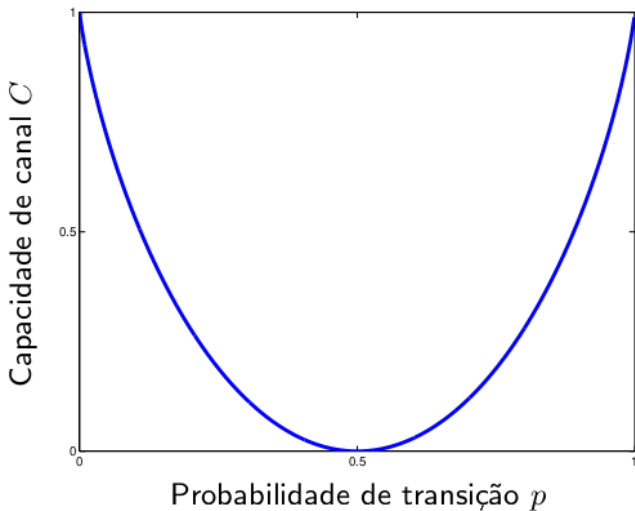
✓ Logo

$$\begin{aligned} C &= 1 + p \cdot \log[p] + (1 - p) \cdot \log[1 - p] \\ &= 1 - \mathcal{H}(p) \end{aligned} \tag{107}$$

$\log \Rightarrow \log_2$ por se tratar de bits de informação.



Capacidade do Canal Binário Simétrico - BSC



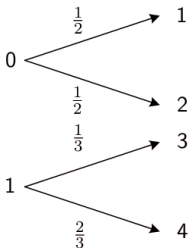
Capacidade do Canal Binário Simétrico - BSC

- ✓ Curva simétrica (esperado!)
- ✓ Quando não há ruído ($p = 0$) a capacidade de canal é máxima, com um bit por uso do canal. Neste mesmo valor, a entropia atinge seu valor mínimo
- ✓ Com a probabilidade de erro $p = 1/2$, a capacidade atinge o menor valor enquanto a entropia atinge o máximo. Nestes casos, o canal é dito ser **sem utilidade** (useless)



Canal BSC - Com saídas com ruído não superpostas

$$C = \max \mathcal{I}(X, Y) = 1 \text{ bit}, \text{ para } p(x) = \left(\frac{1}{2}, \frac{1}{2}\right)$$

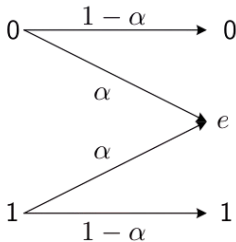


Embora o ruído adicione incerteza, a característica "não superposta" (ou disjunta) significa que, ao observar a saída Y , o receptor consegue determinar com certeza absoluta qual foi o bit de entrada X , resultando em uma informação mútua máxima.



Canal Binário com Apagamento (BEC)

A capacidade de um canal que combina características de um Canal BSC (onde bits podem ser invertidos) e um Canal Binário com Apagamento (BEC) (onde bits podem ser perdidos ou apagados) depende da probabilidade de erro p e da probabilidade de apagamento ϵ (e = apagamento).



$$C_{\max} = \max \mathcal{I}(X, Y) = 1 - \alpha \text{ bits}$$

(108)



Canal Binário com Apagamento (BEC)

Definição e Capacidade

Neste tipo de canal, o transmissor envia 0 ou 1. O receptor pode receber o 0, o 1, ou um símbolo de "apagamento" (chamado geralmente de e ou Null, indicando que o bit foi perdido).

A capacidade C deste canal (em bits por uso do canal) é dada por:

$$C = (1 - \epsilon)(1 - \mathcal{H}(p)) \quad (109)$$

Onde:

- ✓ ϵ é a probabilidade de apagamento ($0 \leq \epsilon \leq 1$).
- ✓ p é a probabilidade de erro (inversão do bit), dado que o bit não foi apagado.
- ✓ $\mathcal{H}$ é a função de entropia binária, definida como:
 $\mathcal{H} = -p \cdot \log(p) - (1 - p) \cdot \log(1 - p)$.



1. Taxa de Informação
 - Estimadores
2. Codificação de Fonte
3. Compressão de Dados
 - Compressão de Huffman
 - Compressão Lempel-Ziv-Welch
4. Capacidade de Canal





TRATAMENTO DE DADOS E COMPRESSÃO

Prof. Dr. Julio César Santos dos Anjos
Dados e Informação

Técnicas de Compressão de Dados

— 27 de maio de 2026